

Propagation

<http://www.byteslounge.com/tutorials/spring-transaction-propagation-tutorial>

<tx:annotation-driven/> is same as @EnableTransactionManagement + @Configuration

Enables Spring's annotation-driven transaction management capability, similar to the support found in Spring's <tx:*> XML namespace. To be used on [@Configuration](#) classes as follows:

```
@Configuration
@EnableTransactionManagement
public class AppConfig {

    @Bean
    public FooRepository fooRepository() {
        // configure and return a class having @Transactional methods
        return new JdbcFooRepository(dataSource());
    }

    @Bean
    public DataSource dataSource() {
        // configure and return the necessary JDBC DataSource
    }

    @Bean
    public PlatformTransactionManager txManager() {
        return new DataSourceTransactionManager(dataSource());
    }
}
```

For reference, the example above can be compared to the following Spring XML configuration:

```
<beans>

    <tx:annotation-driven/>

    <bean id="fooRepository" class="com.foo.JdbcFooRepository">
        <constructor-arg ref="dataSource"/>
    </bean>

    <bean id="dataSource" class="com.vendor.VendorDataSource"/>

    <bean id="transactionManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
        <constructor-arg ref="dataSource"/>
    </bean>

</beans>
```

There is an option to fake Spring bean with just plain Spring features. You need to use @Primary, @Profile and @ActiveProfiles annotations for it.

[I wrote a blog post on the topic.](#)

You can use in memory DB (e.g. H2) to replace real data source. Something like this:

```
@Configuration
public class TestingDataSourceConfig {

    @Bean
```

```

@Primary
public DataSource dataSource() {
    return new EmbeddedDatabaseBuilder()
        .generateUniqueName(true)
        .setType(H2)
        .setScriptEncoding("UTF-8")
        .ignoreFailedDrops(true)
        .addScript("schema.sql")
        .addScripts("user_data.sql", "country_data.sql")
        .build();
}
}

```

REQUIRED behavior

Spring **REQUIRED** behavior means that the same transaction will be used if there is an already opened transaction in the current bean method execution context. If there is no existing transaction the Spring container will create a new one. If multiple methods configured as **REQUIRED** behavior are called in a nested way they will be assigned **distinct logical transactions** but they will all share the **same physical transaction**. In short this means that if an inner method causes a transaction to rollback, the outer method will fail to commit and will also rollback the transaction.

Note: When using declarative transactions, i.e. by using only annotations, and calling methods from the same bean directly (self-invocation), the **@Transactional** annotation will be ignored by the container. If you want to enable transaction management in self-invocations you must configure the transactions using **AspectJ**, but this will not be covered in this tutorial.

REQUIRES_NEW behavior

REQUIRES_NEW behavior means that a new physical transaction will always be created by the container. In other words the inner transaction may commit or rollback independently of the outer transaction, i.e. the outer transaction will not be affected by the inner transaction result: they will run in **distinct physical transactions**.

NESTED behavior

The **NESTED** behavior makes nested Spring transactions to use the same physical transaction but sets savepoints between nested invocations so inner transactions may also rollback independently of outer transactions. This may be familiar to JDBC aware developers as the savepoints are achieved with JDBC savepoints, so this behavior should only be used with Spring JDBC managed transactions ([Spring JDBC transactions example](#)).

MANDATORY behavior

The **MANDATORY** behavior states that an existing opened transaction must already exist. If not an exception will be thrown by the container.

NEVER behavior

Execute non-transactionally, throw an exception if a transaction exists.

The **NEVER** behavior states that an existing opened transaction must **not** already exist. If a transaction exists an exception will be thrown by the container.

NOT_SUPPORTED behavior

Execute non-transactionally, suspend the current transaction if one exists.

The **NOT_SUPPORTED** behavior will execute outside of the scope of any transaction. If an opened transaction already exists it will be paused.

SUPPORTS behavior

The **SUPPORTS** behavior will execute in the scope of a transaction if an opened transaction already exists. If there isn't an already opened transaction the method will execute anyway but in a non-transactional way.

<http://forum.spring.io/forum/spring-projects/data/7372-propagation-nested-versus-propagation-requires-new>

PROPAGATION_REQUIRES_NEW will suspend the current transaction (if one exists) and start a new one. They are independent and a rollback shouldn't affect the first transaction.

PROPAGATION_NESTED will start a nested transaction (if one already exists, otherwise it behaves like PROPAGATION_REQUIRED). I haven't used this, but I expect you can rollback either the nested transaction, or both the nested and containing transaction. The Tx manager (e.g. [DataSourceTransactionManager](#)) must use the JDBC 3.0 Savepoint API (and obviously the DB and driver must support nested transactions).

PROPAGATION_REQUIRES_NEW starts a new, independent "inner" transaction for the given scope. This transaction will be committed or rolled back completely independent from the outer transaction, having its own isolation scope, its own set of locks, etc. The outer transaction will get suspended at the beginning of the inner one, and resumed once the inner one has completed.

Such independent inner transactions are for example used for id generation through manual sequences, where the access to the sequence table should happen in its own transactions, to keep the lock there as short as possible. The goal there is to avoid tying the sequence locks to the (potentially much longer running) outer transaction, with the sequence lock not getting released before completion of the outer transaction.

PROPAGATION_NESTED on the other hand starts a "nested" transaction, which is a true subtransaction of the existing one. What will happen is that a savepoint will be taken at the start of the nested transaction. If the nested transaction fails, we will roll back to that savepoint. The nested transaction is part of the outer transaction, so it will only be committed at the end of the outer transaction.

Nested transactions essentially allow to try some execution subpaths as subtransactions: rolling back to

the state at the beginning of the failed subpath, continuing with another subpath or with the main execution path there - all within one isolated transaction, and not losing any previous work done within the outer transaction.

For example, consider parsing a very large input file consisting of account transfer blocks: The entire file should essentially be parsed within one transaction, with one single commit at the end. But if a block fails, its transfers need to be rolled back, writing a failure marker somewhere. You could either start over the entire transaction every time a block fails, remembering which blocks to skip - or you mark each block as a nested transaction, only rolling back that specific set of operations, keeping the previous work of the outer transaction. The latter is of course much more efficient, in particular when a block at the end of the file fails.

When the transaction attribute is NEVER, no transaction can be propagated to the method. If the transaction is propagated to method it throw exception. The NOT_SUPPORTED will suspend the current transaction. The transaction will then resume when the "NOT_SUPPORTED" method returns.

Tip 1:

Spring recommends that you only annotate concrete classes (and methods of concrete classes) with the `@Transactional` annotation, as opposed to annotating interfaces. You certainly can place the `@Transactional` annotation on an interface (or an interface method), but this works only as you would expect it to if you are using interface-based proxies. The fact that Java annotations are *not inherited from interfaces* means that if you are using class-based proxies (`proxy-target-class="true"`) or the weaving-based aspect (`mode="aspectj"`), then the transaction settings are not recognized by the proxying and weaving infrastructure, and the object will not be wrapped in a transactional proxy, which would be decidedly *bad*.

Tip 2:

Method visibility and `@Transactional`

When using proxies, you should apply the `@Transactional` annotation only to methods with *public* visibility. If you do annotate protected, private or package-visible methods with the `@Transactional` annotation, no error is raised, but the annotated method does not exhibit the configured transactional settings. Consider the use of AspectJ (see below) if you need to annotate non-public methods.

Tip 3:

In proxy mode (which is the default), only external method calls coming in through the proxy are intercepted. This means that self-invocation, in effect, a

method within the target object calling another method of the target object, will not lead to an actual transaction at runtime even if the invoked method is marked with `@Transactional`.